Certainly! Given the extensive nature of this code, I will explain and structure the relationships between classes, data flow, inheritance, method overrides, and overall functionality to make it comprehensible.

Project Overview

The `TicketManagementSystem` manages tickets using multithreading, where:

1. **Vendors** add tickets.
2. **Customers** retrieve tickets.
3. A `Logger` tracks all operations.
4. The `Configuration` encapsulates system settings, and there's support for both CLI and JavaFX-based GUI interfaces.

Directory Structure and Class Interactions

1. `config` Package

Contains `Configuration.java` for managing system settings.

- **Key Functions**:
  - **Encapsulation**: Configuration parameters like total tickets, release rate, retrieval rate, and capacity are encapsulated.
  - **Persistence**: Saves and loads configurations to/from a `config.json` file using Gson.
- **No Inheritance**.

2. `core` Package

Handles ticket operations and shared logic.

1. **`AbstractTicketHandler.java`**
   - Abstract base class for handling tickets.
   - **Inheritance**: Extended by `Vendor` and `Customer` classes.
   - **Abstract Method**: `handleTickets()` is overridden by subclasses.
2. **`TicketOperation.java`**

- Defines a contract (interface) for ticket operations like adding and removing tickets.
- Implemented by `TicketPool`.

3. **`TicketPool.java`**

   - Core ticket management.
   - **Implements**: `TicketOperation`.
   - **Key Features**:
     - Uses a synchronized list to manage concurrent access.
     - Provides methods for adding and removing tickets (`addTickets`, `removeTicket`).
   - **Shared Resource**: Acts as the central shared data for all threads.

---

3. `exceptions` Package

Handles specific errors, e.g., `InvalidConfigurationException.java`.

---

4. `logging` Package

Manages application logging.

1. **`Logger.java`**:
   - Tracks operations (e.g., tickets added/removed) with timestamps.
   - Logs are written to both the console and a `logs.txt` file.

---

5. `threads` Package

Manages multithreaded components.

1. **`Vendor.java`**

   - Extends `AbstractTicketHandler`.
   - Implements `Runnable` to operate in its own thread.
   - **Behavior**:
     - Generates unique tickets.
     - Adds tickets to the `TicketPool`.
     - Logs operations.
     - Sleeps between operations to simulate periodic addition.

2. **`Customer.java`**

    - Extends `AbstractTicketHandler`.
    - Implements `Runnable`.
    - **Behavior**:
        - Continuously removes tickets from the `TicketPool`.
        - Logs successful and failed retrievals.
        - Sleeps between operations to simulate real-world behavior.

---

6. `ui` Package

Supports user interaction via CLI or GUI.

1. **`CommandLineInterface.java`**

    - Prompts users for configuration via the console.
    - Validates input and logs events.
2. **`JavaFXInterface.java`**

    - JavaFX-based graphical user interface.
    - Allows configuration and system monitoring in a more interactive way.
    - **Key Features**:
        - Input fields for configuration.
        - Buttons to start and stop the system.
        - A live log viewer (`TableView`).

---

7. Main.java

Entry point for the program.

- Coordinates system initialization, starting/stopping threads, and handling exceptions.
- **Switchable UI**: Either CLI (`CommandLineInterface`) or GUI (`JavaFXInterface`) can be used.

Data Flow

Initialization:

1. **User Input**: User provides system settings via CLI/GUI.
2. **Configuration**: Stores input values for use by threads.

Core Workflow:

1. **TicketPool**: Central shared resource, accessed by vendors and customers.
2. **Vendors**:
    - Add tickets periodically to the `TicketPool`.
    - Log each operation via `Logger`.
3. **Customers**:
    - Remove tickets (if available) from the `TicketPool`.
    - Log successes and failures.

Logging:

- Logs are output to:
    1. Console (real-time feedback).
    2. `logs.txt` for persistence.

Termination:

- Threads are stopped gracefully when the system is halted or completes processing.

Inheritance and Method Overriding

1. **Inheritance**:
    - `AbstractTicketHandler` (abstract class):
        - **Extended by**: `Vendor` and `Customer`.
    - `TicketOperation` (interface):
        - **Implemented by**: `TicketPool`.
2. **Overridden Methods**:
    - **`handleTickets`** in `Vendor` and `Customer`:
        - Implements ticket-specific behavior (e.g., adding/removing).
    - **`run`** in `Vendor` and `Customer`:
        - Provides threading logic.

Class Relationships

Data Flow Diagram

plaintext Copy code

```
User Input (CLI)
   ↓
System Configuration
   ↓
TicketPool (Queue)
   ↗              ↖
Vendor Thread      Customer Thread
   ↓                  ↓
 Logger (Logs operations)
   ↓
 Console + File Output (logs.txt)
```

Class Relationships

plaintext Copy code

```
Configuration (encapsulates system parameters)

Logger (handles logging)      <-------------------------+
                                                        |
AbstractTicketHandler (abstract)                        |
   ↑                         ↑                          |
Vendor (adds tickets)        Customer (removes tickets)|
   |                         |                          |
   +---------> TicketPool <---+                         |
         (central shared data structure)                |
                                                        |
CLI/JavaFX (user interface for interaction) -----------+
```

Summary

1. **Core Functionality**: A multithreaded system for managing ticket operations.
2. **Modular Design**:

    - Encapsulation (`Configuration`).
    - Abstraction (`AbstractTicketHandler`).
    - Polymorphism (`TicketOperation` interface).
    - Synchronization (`TicketPool` with synchronized collections).
3. **User Interfaces**: Switchable between CLI and GUI.
4. **Threading**:
    - Independent vendor and customer threads.
    - Safe, synchronized access to shared resources.

Would you like me to focus on any specific component, or provide visual aids like diagrams for better understanding?

Sample Question for Lab Based Test
1. Add Logging to Monitor Ticket Operations

Question:
Modify the Logger class to log all ticket operations (addTickets and removeTicket) in a standardized format that includes:
• The ticket ID.
• The type of operation (add/remove).
• The timestamp of the operation.
Update the TicketPool class to use the Logger for every ticket operation. Write a test scenario to verify the logs.

LOGGER CLASS

```
package logging;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;

public class Logger {
   private static final String LOG_FILE = "\resources\\logs";


   public static void log(String message) {
       String timeStampedMessage = LocalDateTime.now() + ": " + message;
       System.out.println(timeStampedMessage);
       try (BufferedWriter writer = new BufferedWriter(new FileWriter(LOG_FILE,
true))) {
           writer.write(timeStampedMessage);
           writer.newLine();
       } catch (IOException e) {
           e.printStackTrace();
       }
   }

   public static void logTicketOperation(String ticketID, String Operation){
       String message = String.format("Ticket ID :%s | Operation
:%s",ticketID,Operation);
       log(message);
   }
}
```

TICKETPOOL CLASS

```
package core;
import logging.Logger;

import java.util.Collections;
import java.util.LinkedList;
import java.util.List;
public class TicketPool implements TicketOperation {
   private final List<String> tickets = Collections.synchronizedList(new
LinkedList<>());
   @Override
   public synchronized void addTickets(String ticket) {
       tickets.add(ticket);
       Logger.logTicketOperation(ticket,"add");
   }
   @Override
```

```
    public synchronized String removeTicket() {
        if (!tickets.isEmpty()){
            String ticket = tickets.remove(0);
            Logger.logTicketOperation(ticket,"remove");
            return ticket;
        }
        else {
            Logger.log("No tickets available to remove");
            return null;
        }
    }
    public int getTicketCount() {
        return tickets.size();
    }
}
```

TEST FILE WRITTEN TO TEST IT

```
package core;

public class TicketPoolTest {
    public static void main(String[] args){
        TicketPool ticketpool = new TicketPool();

        ticketpool.addTickets("Ticket-001");
        ticketpool.addTickets("Ticket-002");

        System.out.println("Removed :"+ticketpool.removeTicket());
        System.out.println("Removed :"+ticketpool.removeTicket());


        System.out.println("Removed :"+ticketpool.removeTicket());
    }
}
```

Updated code for each question

1. Logging to Monitor Ticket Operations

Updated Logger Class

java

Copy code

```
package logging;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;

public class Logger {
    private static final String LOG_FILE = "resources/logs.txt";

    public static void log(String message) {
        String timeStampedMessage = LocalDateTime.now() + ": " +
message;
        System.out.println(timeStampedMessage);
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(LOG_FILE, true))) {
            writer.write(timeStampedMessage);
            writer.newLine();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void logTicketOperation(String operation,
String ticketId) {
```

```
        log(operation + " Ticket ID: " + ticketId + " at " +
LocalDateTime.now());
    }
}
```

Updated TicketPool Class

java

Copy code

```
package core;

import logging.Logger;

import java.util.Collections;
import java.util.LinkedList;
import java.util.List;

public class TicketPool implements TicketOperation {
    private final List<String> tickets =
Collections.synchronizedList(new LinkedList<>());

    @Override
    public synchronized void addTickets(String ticket) {
        tickets.add(ticket);
        Logger.logTicketOperation("Added", ticket);
    }

    @Override
    public synchronized String removeTicket() {
        String ticket = tickets.isEmpty() ? null :
tickets.remove(0);
        if (ticket != null) {
            Logger.logTicketOperation("Removed", ticket);
        }
        return ticket;
    }
```

```
    public int getTicketCount() {
        return tickets.size();
    }
}
```

Test for Ticket Operation Logging

java

Copy code

```
package tests;

import core.TicketPool;
import logging.Logger;

public class TicketOperationTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();

        // Simulating ticket operations
        pool.addTickets("Ticket-1");
        pool.addTickets("Ticket-2");
        pool.removeTicket();
        pool.removeTicket();
        pool.removeTicket(); // Attempting to remove when empty

        // Verify logs in "resources/logs.txt"
    }
}
```

---

2. Extending Customer Functionality with Interfaces

TicketRetrievalStrategy Interface

java

Copy code

```
package strategy;

import core.TicketOperation;

public interface TicketRetrievalStrategy {
    String retrieveTicket(TicketOperation ticketPool);
}
```

Concrete Implementations

java

Copy code

```
package strategy;

import core.TicketOperation;

public class PriorityRetrieval implements
TicketRetrievalStrategy {
    @Override
    public String retrieveTicket(TicketOperation ticketPool) {
        // Custom priority-based logic
        return ticketPool.removeTicket();
    }
}

public class IDRetrieval implements TicketRetrievalStrategy {
    @Override
    public String retrieveTicket(TicketOperation ticketPool) {
        // ID-based retrieval logic
        return ticketPool.removeTicket();
    }
}
```

Updated Customer Class

java

Copy code

```
package threads;

import core.AbstractTicketHandler;
import core.TicketOperation;
import logging.Logger;
import strategy.TicketRetrievalStrategy;

public class Customer extends AbstractTicketHandler implements
Runnable {
    private final TicketRetrievalStrategy strategy;

    public Customer(TicketOperation ticketPool,
TicketRetrievalStrategy strategy) {
        super(ticketPool);
        this.strategy = strategy;
    }

    @Override
    public void run() {
        while (true) {
            String ticket = strategy.retrieveTicket(ticketPool);
            if (ticket != null) {
                Logger.log("Customer retrieved: " + ticket);
            } else {
                Logger.log("Customer found no tickets
available.");
                break;
            }
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                Logger.log("Customer interrupted.");
                break;
            }
        }
```

```
    }

    @Override
    public void handleTickets() {
        run();
    }
}
```

Test for Retrieval Strategies

java

Copy code

```
package tests;

import core.TicketPool;
import strategy.IDRetrieval;
import strategy.PriorityRetrieval;
import threads.Customer;

public class CustomerTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTickets("Ticket-1");
        pool.addTickets("Ticket-2");

        // Test Priority Retrieval
        Customer priorityCustomer = new Customer(pool, new
PriorityRetrieval());
        new Thread(priorityCustomer).start();

        // Test ID Retrieval
        Customer idCustomer = new Customer(pool, new
IDRetrieval());
        new Thread(idCustomer).start();
    }
}
```

---

3. Exception Handling for Invalid Configurations

Custom Exception

java
Copy code

```
package exceptions;

public class InvalidConfigurationException extends Exception {
    public InvalidConfigurationException(String message) {
        super(message);
    }
}
```

Updated Configuration Class

java
Copy code

```
package config;

import exceptions.InvalidConfigurationException;

public class Configuration {
    private int totalTickets;
    private int ticketReleaseRate;
    private int customerRetrievalRate;
    private int maxTicketCapacity;

    public Configuration(int totalTickets, int
ticketReleaseRate, int customerRetrievalRate, int
maxTicketCapacity) throws InvalidConfigurationException {
        if (totalTickets <= 0 || ticketReleaseRate <= 0 ||
customerRetrievalRate <= 0 || maxTicketCapacity <= 0) {
            throw new InvalidConfigurationException("All
configuration values must be positive.");
```

```
        }
        this.totalTickets = totalTickets;
        this.ticketReleaseRate = ticketReleaseRate;
        this.customerRetrievalRate = customerRetrievalRate;
        this.maxTicketCapacity = maxTicketCapacity;
    }

    // Getters and file handling remain unchanged
}
```

Test for Invalid Configuration

java

Copy code

```
package tests;

import config.Configuration;
import exceptions.InvalidConfigurationException;

public class ConfigurationTest {
    public static void main(String[] args) {
        try {
            Configuration config = new Configuration(-1, 10, 5,
20);
        } catch (InvalidConfigurationException e) {
            System.out.println("Exception caught: " +
e.getMessage());
        }
    }
}
```

---

4. Enhancing Vendor Behavior Using Polymorphism

FastVendor and SlowVendor

java

Copy code

```
package threads;

import core.TicketOperation;

public class FastVendor extends Vendor {
    public FastVendor(TicketOperation ticketPool, int
ticketReleaseRate) {
        super(ticketPool, ticketReleaseRate * 2); // Double rate
    }
}

public class SlowVendor extends Vendor {
    public SlowVendor(TicketOperation ticketPool, int
ticketReleaseRate) {
        super(ticketPool, ticketReleaseRate / 2); // Half rate
    }
}
```

Test for Vendor Polymorphism

java

Copy code

```
package tests;

import core.TicketPool;
import threads.FastVendor;
import threads.SlowVendor;

public class VendorTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();

        Thread fastVendor = new Thread(new FastVendor(pool, 5));
        Thread slowVendor = new Thread(new SlowVendor(pool, 5));
```

```
        fastVendor.start();
        slowVendor.start();
    }
}
```

---

5. Periodic Ticket Statistics Reporting

Statistics Reporter

java

Copy code

```
package threads;

import core.TicketPool;
import logging.Logger;

public class StatisticsReporter implements Runnable {
    private final TicketPool ticketPool;

    public StatisticsReporter(TicketPool ticketPool) {
        this.ticketPool = ticketPool;
    }

    @Override
    public void run() {
        while (true) {
            Logger.log("Statistics Report:");
            Logger.log("Total tickets in pool: " +
ticketPool.getTicketCount());
            try {
                Thread.sleep(5000); // Report every 5 seconds
            } catch (InterruptedException e) {
                Logger.log("Statistics Reporter interrupted.");
                break;
            }
```

```
        }
    }
}
```

Test for Statistics Reporting

java

Copy code

```
package tests;

import core.TicketPool;
import threads.StatisticsReporter;

public class StatisticsTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTickets("Ticket-1");
        pool.addTickets("Ticket-2");

        Thread statsThread = new Thread(new
StatisticsReporter(pool));
        statsThread.start();
    }
}
```

6) JavaFX GUI

Updated JavaFX Interface

JavaFXInterface.java

java

Copy code

```
package ui;

import config.Configuration;
import core.TicketPool;
import javafx.application.Application;
```

```
import javafx.application.Platform;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;
import logging.Logger;
import threads.Customer;
import threads.StatisticsReporter;
import threads.Vendor;

import java.util.Timer;
import java.util.TimerTask;

public class JavaFXInterface extends Application {

    private TextField totalTicketsField;
    private TextField ticketReleaseRateField;
    private TextField customerRetrievalRateField;
    private TextField maxTicketCapacityField;
    private Label statusLabel;
    private ListView<String> ticketListView;
    private TicketPool ticketPool;
    private Thread vendorThread;
    private Thread customerThread;
    private Thread statsThread;

    @Override
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Dynamic Ticket Management System");

        // Create a grid layout
        GridPane gridPane = new GridPane();
        gridPane.setVgap(10);
        gridPane.setHgap(10);

        // Configuration input fields
        totalTicketsField = new TextField();
        ticketReleaseRateField = new TextField();
```

```
        customerRetrievalRateField = new TextField();
        maxTicketCapacityField = new TextField();

        gridPane.add(new Label("Total Tickets:"), 0, 0);
        gridPane.add(totalTicketsField, 1, 0);
        gridPane.add(new Label("Ticket Release Rate:"), 0, 1);
        gridPane.add(ticketReleaseRateField, 1, 1);
        gridPane.add(new Label("Customer Retrieval Rate:"), 0, 2);
        gridPane.add(customerRetrievalRateField, 1, 2);
        gridPane.add(new Label("Max Ticket Capacity:"), 0, 3);
        gridPane.add(maxTicketCapacityField, 1, 3);

        // Buttons for start and stop
        Button startButton = new Button("Start");
        Button stopButton = new Button("Stop");
        gridPane.add(startButton, 0, 4);
        gridPane.add(stopButton, 1, 4);

        // Status label
        statusLabel = new Label("System Status: Stopped");
        gridPane.add(statusLabel, 0, 5, 2, 1);

        // Ticket ListView for dynamic visualization
        ticketListView = new ListView<>();
        ticketListView.setPlaceholder(new Label("No tickets in the
pool."));
        gridPane.add(new Label("Ticket Pool:"), 0, 6);
        gridPane.add(ticketListView, 0, 7, 2, 1);

        // Start button action
        startButton.setOnAction(event -> {
            try {
                startSystem();
            } catch (Exception e) {
                Logger.log("Error starting system: " +
e.getMessage());
                updateStatus("Error: Check input values.");
            }
```

```
        });

        // Stop button action
        stopButton.setOnAction(event -> stopSystem());

        // Create a scene and display the stage
        Scene scene = new Scene(gridPane, 600, 400);
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    private void startSystem() throws Exception {
        // Validate and parse inputs
        int totalTickets = validateInput(totalTicketsField.getText(),
"Total Tickets");
        int ticketReleaseRate =
validateInput(ticketReleaseRateField.getText(), "Ticket Release
Rate");
        int customerRetrievalRate =
validateInput(customerRetrievalRateField.getText(), "Customer
Retrieval Rate");
        int maxTicketCapacity =
validateInput(maxTicketCapacityField.getText(), "Max Ticket
Capacity");

        // Initialize ticket pool and configuration
        ticketPool = new TicketPool();
        Configuration config = new Configuration(totalTickets,
ticketReleaseRate, customerRetrievalRate, maxTicketCapacity);

        // Start vendor and customer threads
        vendorThread = new Thread(new Vendor(ticketPool,
config.getTicketReleaseRate()));
        customerThread = new Thread(new Customer(ticketPool, ticket ->
ticketPool.removeTicket()));
        statsThread = new Thread(new StatisticsReporter(ticketPool));

        vendorThread.start();
```

```
        customerThread.start();
        statsThread.start();

        // Start real-time ticket pool updates
        startDynamicVisualization();

        updateStatus("System Running...");
    }

    private void stopSystem() {
        if (vendorThread != null && customerThread != null) {
            vendorThread.interrupt();
            customerThread.interrupt();
            statsThread.interrupt();
        }
        updateStatus("System Stopped.");
    }

    private void startDynamicVisualization() {
        Timer timer = new Timer();
        timer.schedule(new TimerTask() {
            @Override
            public void run() {
                Platform.runLater(() -> {
                    ticketListView.getItems().clear();

ticketListView.getItems().addAll(ticketPool.getTickets());
                });
            }
        }, 0, 1000); // Update every second
    }

    private void updateStatus(String status) {
        Platform.runLater(() -> statusLabel.setText("System Status: "
+ status));
    }
```

```
    private int validateInput(String input, String fieldName) throws
Exception {
        try {
            int value = Integer.parseInt(input);
            if (value <= 0) {
                throw new Exception(fieldName + " must be positive.");
            }
            return value;
        } catch (NumberFormatException e) {
            throw new Exception(fieldName + " must be a valid
integer.");
        }
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

ChatGPT sample questions

1. Add Priority-Based Ticket Management

Question:
Modify the `TicketPool` to support priority-based ticket management. Tickets with higher priority should be retrieved first.

Solution:

- Use a `PriorityQueue` to manage tickets based on priority.
- Each ticket should have an associated priority value.

Code Implementation:

java
Copy code

```
package core;
```

```java
import java.util.PriorityQueue;

public class TicketPool {
    private final PriorityQueue<Ticket> ticketQueue = new
PriorityQueue<>();

    public synchronized void addTicket(String ticketId, int priority)
{
        ticketQueue.add(new Ticket(ticketId, priority));
    }

    public synchronized String removeTicket() {
        Ticket ticket = ticketQueue.poll();
        return ticket != null ? ticket.getTicketId() : null;
    }
}
```

Test Case:

java
Copy code

```java
package tests;

import core.TicketPool;

public class PriorityTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();

        pool.addTicket("Ticket-1", 3);
        pool.addTicket("Ticket-2", 1);
        pool.addTicket("Ticket-3", 2);

        System.out.println(pool.removeTicket()); // Should print
Ticket-2
        System.out.println(pool.removeTicket()); // Should print
Ticket-3
```

```
        System.out.println(pool.removeTicket()); // Should print
Ticket-1
    }
}
```

2. Implement a Singleton Logger

Question:
Refactor the `Logger` class to use the Singleton pattern for ensuring a single instance.

Code Implementation:

java
Copy code

```
package logging;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Logger {
    private static Logger instance;
    private static final String LOG_FILE = "resources/logs.txt";

    private Logger() {}

    public static synchronized Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void log(String message) {
        System.out.println(message);
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(LOG_FILE, true))) {
```

```
            writer.write(message);
            writer.newLine();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Test Case:

java
Copy code

```
package tests;

import logging.Logger;

public class LoggerSingletonTest {
    public static void main(String[] args) {
        Logger logger = Logger.getInstance();
        logger.log("This is a test log.");
    }
}
```

3. Add a Ticket Expiry Feature

Question:
Tickets should expire after a fixed duration. Modify the `TicketPool` to remove expired tickets automatically.

Solution:

- Use a `ScheduledExecutorService` to periodically check and remove expired tickets.

Code Implementation:

java
Copy code

```
package core;

import java.util.concurrent.ConcurrentHashMap;
```

```
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

public class TicketPool {
    private final ConcurrentHashMap<String, Long> tickets = new
ConcurrentHashMap<>();

    public TicketPool() {
        ScheduledExecutorService scheduler =
Executors.newScheduledThreadPool(1);
        scheduler.scheduleAtFixedRate(() -> {
            long currentTime = System.currentTimeMillis();
            tickets.entrySet().removeIf(entry -> currentTime -
entry.getValue() > 5000); // 5s expiry
        }, 0, 1, TimeUnit.SECONDS);
    }

    public synchronized void addTicket(String ticketId) {
        tickets.put(ticketId, System.currentTimeMillis());
    }

    public synchronized String removeTicket() {
        return tickets.keySet().stream().findFirst().orElse(null);
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;

public class TicketExpiryTest {
    public static void main(String[] args) throws InterruptedException
{
        TicketPool pool = new TicketPool();
```

```
        pool.addTicket("Ticket-1");
        Thread.sleep(6000); // Wait for ticket expiry
        System.out.println(pool.removeTicket()); // Should print null
    }
}
```

4. Extend Customer with Async Callback

Question:
Allow customers to specify a callback function that executes after a ticket is retrieved.

Code Implementation:

java
Copy code

```
package threads;

import core.TicketOperation;

public class AsyncCustomer implements Runnable {
    private final TicketOperation ticketPool;
    private final Runnable callback;

    public AsyncCustomer(TicketOperation ticketPool, Runnable
callback) {
        this.ticketPool = ticketPool;
        this.callback = callback;
    }

    @Override
    public void run() {
        String ticket = ticketPool.removeTicket();
        if (ticket != null) {
            System.out.println("Retrieved ticket: " + ticket);
            callback.run();
        } else {
            System.out.println("No tickets available.");
        }
    }
```

```
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import threads.AsyncCustomer;

public class AsyncCustomerTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket("Ticket-1");

        AsyncCustomer customer = new AsyncCustomer(pool, () ->
System.out.println("Callback executed!"));
        new Thread(customer).start();
    }
}
```

5. Add Batch Ticket Retrieval

Question:
Allow customers to retrieve multiple tickets in a single operation.

Code Implementation:

java
Copy code

```
package core;

import java.util.ArrayList;
import java.util.List;

public class TicketPool {
    private final List<String> tickets = new ArrayList<>();
```

```
    public synchronized void addTicket(String ticket) {
        tickets.add(ticket);
    }

    public synchronized List<String> removeBatch(int count) {
        List<String> batch = new ArrayList<>();
        for (int i = 0; i < count && !tickets.isEmpty(); i++) {
            batch.add(tickets.remove(0));
        }
        return batch;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;

import java.util.List;

public class BatchRetrievalTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket("Ticket-1");
        pool.addTicket("Ticket-2");
        pool.addTicket("Ticket-3");

        List<String> batch = pool.removeBatch(2);
        System.out.println(batch); // Should print [Ticket-1,
Ticket-2]
    }
}
```

7. Add Vendor Priority Support

Question:
Modify the system to allow vendors to have a priority level. Vendors with higher priority should release tickets first.

Solution:
Use a `PriorityBlockingQueue` for managing vendor threads based on their priority.

Code Implementation:

java
Copy code

```
package threads;

import java.util.concurrent.PriorityBlockingQueue;

public class VendorManager {
    private final PriorityBlockingQueue<Vendor> vendorQueue = new
PriorityBlockingQueue<>();

    public void addVendor(Vendor vendor) {
        vendorQueue.add(vendor);
    }

    public void startVendors() {
        while (!vendorQueue.isEmpty()) {
            Vendor vendor = vendorQueue.poll();
            new Thread(vendor).start();
        }
    }
}
```

Test Case:

java
Copy code

```
package tests;
```

```
import core.TicketPool;
import threads.Vendor;
import threads.VendorManager;

public class VendorPriorityTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        VendorManager manager = new VendorManager();

        manager.addVendor(new Vendor(pool, 5)); // Default priority
        manager.addVendor(new Vendor(pool, 10)); // Higher priority

        manager.startVendors();
    }
}
```

8. Add Ticket Type Filtering

Question:
Support filtering tickets by type (e.g., VIP, Regular) during retrieval.

Solution:
Add a `type` field to tickets and a filtering mechanism.

Code Implementation:

java
Copy code

```
package core;

public class Ticket {
    private final String id;
    private final String type;

    public Ticket(String id, String type) {
        this.id = id;
        this.type = type;
    }
```

```
    public String getId() {
        return id;
    }

    public String getType() {
        return type;
    }
}
```

Test Case:

java
Copy code
```
package tests;

import core.TicketPool;
import core.Ticket;

public class TicketTypeFilterTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket(new Ticket("Ticket-1", "VIP"));
        pool.addTicket(new Ticket("Ticket-2", "Regular"));

        System.out.println(pool.retrieveTicketByType("VIP")); //
Should return Ticket-1
    }
}
```

9. Add Logging Level Support

Question:
Extend the `Logger` to support different logging levels (INFO, DEBUG, ERROR).

Code Implementation:

java
Copy code
```
package logging;
```

```
public enum LogLevel {
    INFO, DEBUG, ERROR
}

public class Logger {
    private LogLevel currentLevel = LogLevel.INFO;

    public void setLogLevel(LogLevel level) {
        this.currentLevel = level;
    }

    public void log(String message, LogLevel level) {
        if (level.ordinal() >= currentLevel.ordinal()) {
            System.out.println(level + ": " + message);
        }
    }
}
```

Test Case:

java
Copy code

```
package tests;

import logging.Logger;
import logging.LogLevel;

public class LoggerLevelTest {
    public static void main(String[] args) {
        Logger logger = new Logger();
        logger.setLogLevel(LogLevel.DEBUG);

        logger.log("This is an INFO log.", LogLevel.INFO);
        logger.log("This is a DEBUG log.", LogLevel.DEBUG);
        logger.log("This is an ERROR log.", LogLevel.ERROR);
    }
}
```

11. GUI Enhancement: Display Vendor and Customer Count Dynamically

Question:
Add labels to the GUI to dynamically display the number of active vendors and customers.

Solution:
Use `Label` components for vendor and customer counts and update them using `Platform.runLater()` whenever their count changes.

Code Implementation

java
Copy code

```
package ui;

import javafx.application.Application;
import javafx.application.Platform;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class VendorCustomerCountGUI extends Application {

    private Label vendorCountLabel;
    private Label customerCountLabel;
    private int vendorCount = 0;
    private int customerCount = 0;

    @Override
    public void start(Stage primaryStage) {
        vendorCountLabel = new Label("Active Vendors: " +
vendorCount);
        customerCountLabel = new Label("Active Customers: " +
customerCount);

        VBox root = new VBox(10, vendorCountLabel,
customerCountLabel);
        Scene scene = new Scene(root, 300, 200);
```

```
        primaryStage.setScene(scene);
        primaryStage.setTitle("Vendor & Customer Count");
        primaryStage.show();

        // Simulate vendor/customer count changes
        new Thread(() -> updateVendorCount(5)).start();
        new Thread(() -> updateCustomerCount(3)).start();
    }

    private void updateVendorCount(int newCount) {
        try {
            Thread.sleep(2000); // Simulate delay
            vendorCount = newCount;
            Platform.runLater(() -> vendorCountLabel.setText("Active
Vendors: " + vendorCount));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    private void updateCustomerCount(int newCount) {
        try {
            Thread.sleep(2000); // Simulate delay
            customerCount = newCount;
            Platform.runLater(() -> customerCountLabel.setText("Active
Customers: " + customerCount));
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

12. Periodic Ticket Count Refresh in GUI

Question:
Modify the GUI to display the current ticket count in real time.

Solution:
Add a `Label` for the ticket count and use a `Timer` to update the label every second.

Code Implementation

java
Copy code

```
package ui;

import javafx.application.Application;
import javafx.application.Platform;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

import java.util.Timer;
import java.util.TimerTask;

public class TicketCountRefreshGUI extends Application {

    private Label ticketCountLabel;
    private int ticketCount = 0;

    @Override
    public void start(Stage primaryStage) {
        ticketCountLabel = new Label("Tickets Available: " +
ticketCount);

        VBox root = new VBox(10, ticketCountLabel);
        Scene scene = new Scene(root, 300, 150);
        primaryStage.setScene(scene);
```

```
        primaryStage.setTitle("Ticket Count Refresh");
        primaryStage.show();

        // Periodically update ticket count
        Timer timer = new Timer();
        timer.schedule(new TimerTask() {
            @Override
            public void run() {
                Platform.runLater(() -> {
                    ticketCount += 5; // Simulate ticket additions
                    ticketCountLabel.setText("Tickets Available: " +
ticketCount);
                });
            }
        }, 0, 1000); // Update every second
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

13. Implement Factory Pattern for Vendors

Question:
Use the Factory pattern to create different types of vendors (`FastVendor`, `SlowVendor`, `RegularVendor`).

Code Implementation

java
Copy code

```
package factory;

import core.TicketPool;
import threads.FastVendor;
import threads.SlowVendor;
import threads.Vendor;
```

```
public class VendorFactory {

    public static Vendor createVendor(String type, TicketPool pool,
int ticketRate) {
        return switch (type.toLowerCase()) {
            case "fast" -> new FastVendor(pool, ticketRate);
            case "slow" -> new SlowVendor(pool, ticketRate);
            default -> new Vendor(pool, ticketRate);
        };
    }
}
```

Test Case

java
Copy code

```
package tests;

import core.TicketPool;
import factory.VendorFactory;
import threads.Vendor;

public class VendorFactoryTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();

        Vendor fastVendor = VendorFactory.createVendor("fast", pool,
10);
        Vendor slowVendor = VendorFactory.createVendor("slow", pool,
5);
        Vendor regularVendor = VendorFactory.createVendor("regular",
pool, 7);

        new Thread(fastVendor).start();
        new Thread(slowVendor).start();
        new Thread(regularVendor).start();
    }
}
```

14. Add Ticket Reservation Feature

Question:
Implement a feature where tickets can be reserved for a certain period before being sold.

Code Implementation

java
Copy code

```java
package core;

import java.util.HashMap;
import java.util.Map;

public class TicketPool {
    private final Map<String, Long> reservedTickets = new HashMap<>();

    public synchronized boolean reserveTicket(String ticketId) {
        if (reservedTickets.containsKey(ticketId)) {
            return false; // Already reserved
        }
        reservedTickets.put(ticketId, System.currentTimeMillis());
        return true;
    }

    public synchronized boolean isTicketReserved(String ticketId) {
        return reservedTickets.containsKey(ticketId);
    }
}
```

Test Case

java
Copy code

```java
package tests;

import core.TicketPool;
```

```
public class ReservationTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();

        pool.reserveTicket("Ticket-1");
        System.out.println(pool.isTicketReserved("Ticket-1")); // true
    }
}
```

15. Implement Observer Pattern for Ticket Reservations

Question:
Modify the system to notify observers when tickets are reserved or released.

Solution:

- Use the **Observer** pattern to notify components about ticket reservation changes.
- Create a `ReservationObserver` interface and implement concrete observers (e.g., `LoggerObserver`).

Code Implementation

ReservationObserver Interface

java
Copy code

```
package observer;

public interface ReservationObserver {
    void update(String message);
}
```

TicketPool with Reservation Observers

java
Copy code

```
package core;
```

```
import observer.ReservationObserver;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ConcurrentHashMap;

public class TicketPool {
    private final ConcurrentHashMap<String, Long> reservedTickets =
new ConcurrentHashMap<>();
    private final List<ReservationObserver> observers = new
ArrayList<>();

    public synchronized boolean reserveTicket(String ticketId) {
        if (reservedTickets.containsKey(ticketId)) {
            return false; // Already reserved
        }
        reservedTickets.put(ticketId, System.currentTimeMillis());
        notifyObservers("Reserved: " + ticketId);
        return true;
    }

    public synchronized boolean releaseTicket(String ticketId) {
        if (reservedTickets.remove(ticketId) != null) {
            notifyObservers("Released: " + ticketId);
            return true;
        }
        return false;
    }

    public synchronized void addObserver(ReservationObserver observer)
{
        observers.add(observer);
    }

    public synchronized void removeObserver(ReservationObserver
observer) {
        observers.remove(observer);
    }
```

```
    private void notifyObservers(String message) {
        for (ReservationObserver observer : observers) {
            observer.update(message);
        }
    }
}
```

LoggerObserver Implementation

java
Copy code

```
package observer;

import logging.Logger;

public class LoggerObserver implements ReservationObserver {
    @Override
    public void update(String message) {
        Logger.getInstance().log("Reservation Update: " + message);
    }
}
```

Test Case

java
Copy code

```
package tests;

import core.TicketPool;
import observer.LoggerObserver;

public class ReservationObserverTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        LoggerObserver loggerObserver = new LoggerObserver();
        pool.addObserver(loggerObserver);
```

```
        pool.reserveTicket("Ticket-1");
        pool.releaseTicket("Ticket-1");
    }
}
```

16. Add Ticket Search Feature

Question:
Enable searching for tickets by ID or type in the `TicketPool`.

Solution:

- Add methods for searching tickets by ID or type in the `TicketPool`.
- Use streams for efficient filtering.

Code Implementation

java
Copy code

```
package core;

import java.util.List;
import java.util.stream.Collectors;

public class TicketPool {
    private final List<Ticket> tickets = new java.util.LinkedList<>();

    public synchronized List<Ticket> searchTicketsByType(String type)
{
        return tickets.stream().filter(ticket ->
ticket.getType().equalsIgnoreCase(type)).collect(Collectors.toList());
    }

    public synchronized Ticket searchTicketById(String id) {
        return tickets.stream().filter(ticket ->
ticket.getId().equalsIgnoreCase(id)).findFirst().orElse(null);
    }
```

```
}
```

Test Case

java
Copy code

```
package tests;

import core.Ticket;
import core.TicketPool;

import java.util.List;

public class TicketSearchTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket(new Ticket("Ticket-1", "VIP"));
        pool.addTicket(new Ticket("Ticket-2", "Regular"));

        Ticket vipTicket = pool.searchTicketById("Ticket-1");
        System.out.println("Found Ticket: " + vipTicket.getId());

        List<Ticket> regularTickets =
pool.searchTicketsByType("Regular");
        System.out.println("Regular Tickets: " +
regularTickets.size());
    }
}
```

---

17. GUI Enhancement: Add Ticket Purchase History

Question:
Add a `ListView` in the GUI to display the history of tickets purchased by customers.

Solution:

- Maintain a purchase history in the `TicketPool`.
- Update the `ListView` dynamically as tickets are purchased.

Code Implementation

java
Copy code

```
package ui;

import core.TicketPool;
import javafx.application.Application;
import javafx.application.Platform;
import javafx.scene.Scene;
import javafx.scene.control.ListView;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class PurchaseHistoryGUI extends Application {

    private ListView<String> purchaseHistoryListView;
    private TicketPool ticketPool;

    @Override
    public void start(Stage primaryStage) {
        ticketPool = new TicketPool();
        purchaseHistoryListView = new ListView<>();

        VBox root = new VBox(10, purchaseHistoryListView);
        Scene scene = new Scene(root, 300, 200);

        primaryStage.setScene(scene);
        primaryStage.setTitle("Purchase History");
        primaryStage.show();

        // Simulate ticket purchases
        new Thread(() -> simulatePurchases()).start();
    }
```

```
    private void simulatePurchases() {
        try {
            Thread.sleep(1000);
            ticketPool.addTicket("Ticket-1");
            Thread.sleep(1000);
            ticketPool.removeTicket("Ticket-1");
            updateHistory("Purchased: Ticket-1");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    private void updateHistory(String message) {
        Platform.runLater(() ->
purchaseHistoryListView.getItems().add(message));
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

---

18. Add Statistics Dashboard

Question:
Create a GUI dashboard to display ticket statistics like total tickets added, removed, and current pool size.

Code Implementation

java
Copy code

```
package ui;

import core.TicketPool;
import javafx.application.Application;
import javafx.application.Platform;
```

```
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

import java.util.Timer;
import java.util.TimerTask;

public class StatisticsDashboard extends Application {

    private Label totalAddedLabel;
    private Label totalRemovedLabel;
    private Label currentPoolSizeLabel;
    private TicketPool ticketPool;

    @Override
    public void start(Stage primaryStage) {
        ticketPool = new TicketPool();

        totalAddedLabel = new Label("Total Tickets Added: 0");
        totalRemovedLabel = new Label("Total Tickets Removed: 0");
        currentPoolSizeLabel = new Label("Current Pool Size: 0");

        VBox root = new VBox(10, totalAddedLabel, totalRemovedLabel,
currentPoolSizeLabel);
        Scene scene = new Scene(root, 400, 200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Ticket Statistics Dashboard");
        primaryStage.show();

        // Simulate ticket operations and refresh statistics
        new Timer().schedule(new TimerTask() {
            @Override
            public void run() {
                Platform.runLater(() -> refreshStatistics());
            }
        }, 0, 2000);
    }
```

```
    private void refreshStatistics() {
        totalAddedLabel.setText("Total Tickets Added: " +
ticketPool.getTotalAdded());
        totalRemovedLabel.setText("Total Tickets Removed: " +
ticketPool.getTotalRemoved());
        currentPoolSizeLabel.setText("Current Pool Size: " +
ticketPool.getCurrentPoolSize());
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

---

19. Implement a Rate Limiter for Vendors

Question:
Ensure vendors cannot release tickets faster than a specified rate.

Code Implementation

java
Copy code

```
package threads;

import core.TicketPool;

public class RateLimitedVendor implements Runnable {
    private final TicketPool ticketPool;
    private final int rateLimit; // Tickets per second

    public RateLimitedVendor(TicketPool ticketPool, int rateLimit) {
        this.ticketPool = ticketPool;
        this.rateLimit = rateLimit;
    }
```

```
    @Override
    public void run() {
        try {
            while (true) {
                ticketPool.addTicket("Ticket-" + System.nanoTime());
                Thread.sleep(1000 / rateLimit); // Enforce rate limit
            }
        } catch (InterruptedException e) {
            System.out.println("Vendor stopped.");
        }
    }
}
```

Test Case

java
Copy code

```
package tests;

import core.TicketPool;
import threads.RateLimitedVendor;

public class RateLimiterTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        Thread vendor = new Thread(new RateLimitedVendor(pool, 5)); //
5 tickets/sec
        vendor.start();
    }
}
```

---

20. Extend Customer Functionality for Group Purchases

Question:
Allow customers to purchase multiple tickets in a single operation.

Code Implementation

java
Copy code

```java
package threads;

import core.TicketPool;

public class GroupCustomer implements Runnable {
    private final TicketPool ticketPool;
    private final int ticketsToBuy;

    public GroupCustomer(TicketPool ticketPool, int ticketsToBuy) {
        this.ticketPool = ticketPool;
        this.ticketsToBuy = ticketsToBuy;
    }

    @Override
    public void run() {
        for (int i = 0; i < ticketsToBuy; i++) {
            String ticket = ticketPool.removeTicket();
            if (ticket != null) {
                System.out.println("Purchased: " + ticket);
            } else {
                System.out.println("No tickets available.");
                break;
            }
        }
    }
}
```

Test Case

java
Copy code

```java
package tests;

import core.TicketPool;
import threads.GroupCustomer;
```

```
public class GroupPurchaseTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket("Ticket-1");
        pool.addTicket("Ticket-2");

        Thread customer = new Thread(new GroupCustomer(pool, 3)); //
Request 3 tickets
        customer.start();
    }
}
```

21. Implement a Thread Pool for Vendors and Customers

Question:
Use a thread pool to manage vendor and customer threads instead of creating new threads for each operation.

Solution:

- Use `ExecutorService` to manage and pool threads for vendors and customers.

Code Implementation:

java
Copy code

```
package threads;

import core.TicketPool;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ThreadPoolManager {
    private final ExecutorService vendorExecutor;
    private final ExecutorService customerExecutor;
```

```
    public ThreadPoolManager(int numVendors, int numCustomers,
TicketPool ticketPool) {
        vendorExecutor = Executors.newFixedThreadPool(numVendors);
        customerExecutor = Executors.newFixedThreadPool(numCustomers);

        // Add vendors and customers to the executor
        for (int i = 0; i < numVendors; i++) {
            vendorExecutor.submit(new Vendor(ticketPool, 5)); // 5
tickets per vendor
        }

        for (int i = 0; i < numCustomers; i++) {
            customerExecutor.submit(new Customer(ticketPool, 3)); // 3
tickets per customer
        }
    }

    public void shutdown() {
        vendorExecutor.shutdown();
        customerExecutor.shutdown();
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import threads.ThreadPoolManager;

public class ThreadPoolTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        ThreadPoolManager threadPoolManager = new ThreadPoolManager(2,
3, pool);
        threadPoolManager.shutdown();
    }
```

```
}
```

---

22. Implement a Time-Based Discount for Tickets

Question:
Introduce a feature that applies a discount to tickets based on how long they've been in the pool. Tickets older than 5 minutes get a 10% discount.

Solution:

- Add a `timestamp` field to the `Ticket` class and calculate the discount when the ticket is retrieved.

Code Implementation:

java
Copy code

```
package core;

import java.time.Instant;

public class Ticket {
    private final String id;
    private final double price;
    private final Instant timestamp;

    public Ticket(String id, double price) {
        this.id = id;
        this.price = price;
        this.timestamp = Instant.now();
    }

    public String getId() {
        return id;
    }

    public double getPrice() {
```

```
        long timeInPool = Instant.now().getEpochSecond() -
timestamp.getEpochSecond();
        if (timeInPool > 300) {  // 5 minutes = 300 seconds
            return price * 0.9;  // 10% discount
        }
        return price;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;

public class TicketDiscountTest {
    public static void main(String[] args) {
        Ticket ticket = new Ticket("Ticket-1", 100.0);
        System.out.println("Original Price: " + ticket.getPrice());
        // Wait for 5 minutes and then check the price again
    }
}
```

---

23. Add Logging for Ticket Reservation Attempts

Question:
Log every attempt to reserve a ticket, whether successful or failed.

Solution:

- Add logging to the `reserveTicket` method to track reservation attempts.

Code Implementation:

java

Copy code

```
package core;

import logging.Logger;

public class TicketPool {

    public synchronized boolean reserveTicket(String ticketId) {
        if (!tickets.contains(ticketId)) {
            Logger.log("Failed Reservation: " + ticketId);
            return false;
        }
        tickets.remove(ticketId);
        Logger.log("Successful Reservation: " + ticketId);
        return true;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;

public class ReservationLoggingTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.reserveTicket("Ticket-1");
    }
}
```

---

24. Implement a Ticket Expiry Notification

Question:
Notify customers when a ticket is about to expire (e.g., 1 minute before expiry).

Solution:

- Use a scheduled task to check for tickets nearing expiry and notify users.

Code Implementation:

java
Copy code

```
package core;

import java.time.Instant;
import java.util.Timer;
import java.util.TimerTask;

public class TicketPool {
    private final Timer timer = new Timer();
    private final List<Ticket> tickets = new ArrayList<>();

    public TicketPool() {
        timer.schedule(new TimerTask() {
            @Override
            public void run() {
                checkTicketExpiry();
            }
        }, 0, 60000);  // Check every minute
    }

    public void addTicket(String ticketId) {
        tickets.add(new Ticket(ticketId, Instant.now()));
    }

    private void checkTicketExpiry() {
        Instant now = Instant.now();
        for (Ticket ticket : tickets) {
            if (ticket.getTimestamp().plusSeconds(300).isBefore(now))
{
                System.out.println("Ticket " + ticket.getId() + " is
about to expire!");
```

```
            }
        }
    }
}
```

Test Case:

java
Copy code
```
package tests;

import core.TicketPool;

public class ExpiryNotificationTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket("Ticket-1");
    }
}
```

---

25. Implement a Discount for Bulk Purchases

Question:
Provide a 15% discount when customers buy more than 5 tickets in one purchase.

Solution:

- Modify the `Customer` class to check for bulk purchases and apply a discount.

Code Implementation:

java
Copy code
```
package threads;

import core.TicketPool;
import java.util.List;
```

```
public class Customer implements Runnable {
    private final TicketPool ticketPool;
    private final int ticketsToPurchase;

    public Customer(TicketPool ticketPool, int ticketsToPurchase) {
        this.ticketPool = ticketPool;
        this.ticketsToPurchase = ticketsToPurchase;
    }

    @Override
    public void run() {
        double totalPrice = 0;
        for (int i = 0; i < ticketsToPurchase; i++) {
            String ticket = ticketPool.removeTicket();
            if (ticket != null) {
                totalPrice += ticketPool.getTicketPrice(ticket);
            }
        }

        if (ticketsToPurchase > 5) {
            totalPrice *= 0.85; // 15% discount for bulk purchase
        }
        System.out.println("Total Price: " + totalPrice);
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import threads.Customer;

public class BulkPurchaseTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket("Ticket-1");
```

```
        pool.addTicket("Ticket-2");
        pool.addTicket("Ticket-3");
        pool.addTicket("Ticket-4");
        pool.addTicket("Ticket-5");
        pool.addTicket("Ticket-6");

        Customer customer = new Customer(pool, 6);
        new Thread(customer).start();
    }
}
```

---

26. Implement Ticket Pool Shuffling

Question:
Implement a feature where tickets in the pool are shuffled to randomize their order before retrieval.

Solution:

- Use `Collections.shuffle()` to randomize the order of tickets in the pool.

Code Implementation:

java
Copy code

```
package core;

import java.util.Collections;
import java.util.List;

public class TicketPool {

    private final List<Ticket> tickets;

    public TicketPool(List<Ticket> tickets) {
        this.tickets = tickets;
    }
```

```
    public synchronized Ticket getRandomTicket() {
        Collections.shuffle(tickets);
        return tickets.isEmpty() ? null : tickets.get(0);
    }
}
```

Test Case:

java
Copy code
```
package tests;

import core.TicketPool;
import core.Ticket;

public class TicketShufflingTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool(List.of(new
Ticket("Ticket-1", "VIP"), new Ticket("Ticket-2", "Regular")));
        Ticket ticket = pool.getRandomTicket();
        System.out.println("Randomly Retrieved Ticket: " +
ticket.getId());
    }
}
```

---

27. Implement Ticket Pool Sorting by Expiry Date

Question:
Sort the tickets by their expiration date before they are retrieved.

Solution:

- Sort the `Ticket` list based on the `expiryDate` field before retrieving tickets.

Code Implementation:

java

Copy code

```
package core;

import java.util.Comparator;
import java.util.List;

public class TicketPool {

    private final List<Ticket> tickets;

    public TicketPool(List<Ticket> tickets) {
        this.tickets = tickets;
    }

    public synchronized Ticket getNextTicket() {
        tickets.sort(Comparator.comparing(Ticket::getExpiryDate));
        return tickets.isEmpty() ? null : tickets.get(0);
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import core.Ticket;

public class TicketSortingTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool(List.of(new
Ticket("Ticket-1", "VIP"), new Ticket("Ticket-2", "Regular")));
        Ticket ticket = pool.getNextTicket();
        System.out.println("Next Ticket (sorted by expiry): " +
ticket.getId());
    }
}
```

---

28. Add Ticket Transfer Feature

Question:
Allow a customer to transfer tickets to another customer.

Solution:

- Add a method to transfer tickets between customers.

Code Implementation:

java
Copy code

```
package core;

import java.util.List;

public class TicketPool {

    public synchronized void transferTicket(Customer fromCustomer,
Customer toCustomer, Ticket ticket) {
        fromCustomer.removeTicket(ticket);
        toCustomer.addTicket(ticket);
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import core.Ticket;
import threads.Customer;

public class TicketTransferTest {
    public static void main(String[] args) {
```

```
        TicketPool pool = new TicketPool();
        Ticket ticket = new Ticket("Ticket-1", "VIP");
        Customer fromCustomer = new Customer(pool, 1);
        Customer toCustomer = new Customer(pool, 0);

        pool.transferTicket(fromCustomer, toCustomer, ticket);
        System.out.println("Ticket transferred.");
    }
}
```

---

29. Add Customer Verification Before Purchase

Question:
Verify customer identity before allowing them to purchase a ticket.

Solution:

- Implement a `verifyCustomer` method before ticket purchase.

Code Implementation:

java
Copy code

```
package threads;

import core.TicketPool;

public class Customer implements Runnable {
    private final TicketPool ticketPool;
    private final String customerId;

    public Customer(TicketPool ticketPool, String customerId) {
        this.ticketPool = ticketPool;
        this.customerId = customerId;
    }

    @Override
```

```
    public void run() {
        if (verifyCustomer()) {
            ticketPool.removeTicket();
            System.out.println("Ticket purchased by " + customerId);
        } else {
            System.out.println("Customer verification failed.");
        }
    }

    private boolean verifyCustomer() {
        // Simulate customer verification
        return !customerId.isEmpty();
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import threads.Customer;

public class CustomerVerificationTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        Customer customer = new Customer(pool, "Customer1");
        new Thread(customer).start();
    }
}
```

---

30. Implement Ticket Price Based on Demand

Question:
Increase the ticket price when demand is high (e.g., more than 10 customers requesting tickets).

Solution:

- Monitor ticket demand and adjust prices accordingly.

Code Implementation:

java
Copy code

```
package core;

import java.util.List;

public class TicketPool {
    private final List<Ticket> tickets;
    private int demandCount = 0;

    public TicketPool(List<Ticket> tickets) {
        this.tickets = tickets;
    }

    public synchronized void addDemand() {
        demandCount++;
        if (demandCount > 10) {
            tickets.forEach(ticket -> ticket.increasePrice(10)); //
Increase price by 10%
        }
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import core.Ticket;
```

```
public class TicketDemandTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool(List.of(new
Ticket("Ticket-1", "VIP")));
        pool.addDemand();
        pool.addDemand();  // Continue to add demand and test price
change logic
    }
}
```

31. Implement Ticket Pool Backup and Restore

Question:
Implement functionality to back up and restore the ticket pool from a file.

Solution:

- Use serialization (or a JSON library) to save and load the ticket pool's state.

Code Implementation:

java
Copy code

```
package core;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class TicketPool {

    private List<Ticket> tickets = new ArrayList<>();

    public void addTicket(Ticket ticket) {
        tickets.add(ticket);
    }

    public void saveToFile(String filename) throws IOException {
```

```
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(filename))) {
            oos.writeObject(tickets);
        }
    }

    public void restoreFromFile(String filename) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(filename))) {
            tickets = (List<Ticket>) ois.readObject();
        }
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import core.Ticket;

public class TicketPoolBackupTest {
    public static void main(String[] args) throws Exception {
        TicketPool pool = new TicketPool();
        pool.addTicket(new Ticket("Ticket-1", "VIP"));

        pool.saveToFile("ticket_pool.dat");

        TicketPool restoredPool = new TicketPool();
        restoredPool.restoreFromFile("ticket_pool.dat");
    }
}
```

---

32. Implement Ticket Price Based on Customer Type

Question:
Implement a feature where ticket prices vary based on the customer type (e.g., VIP, Regular).

Solution:

- Modify the `Customer` class to assign a type, and adjust the ticket price based on that.

Code Implementation:

java
Copy code

```
package core;

public class Customer {
    private final String customerId;
    private final String customerType; // "VIP" or "Regular"

    public Customer(String customerId, String customerType) {
        this.customerId = customerId;
        this.customerType = customerType;
    }

    public double getTicketPrice(Ticket ticket) {
        if ("VIP".equals(customerType)) {
            return ticket.getPrice() * 0.8; // VIP gets a 20% discount
        }
        return ticket.getPrice();
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;
import core.Customer;
```

```
public class CustomerTicketPriceTest {
    public static void main(String[] args) {
        Ticket ticket = new Ticket("Ticket-1", "VIP", 100.0);
        Customer vipCustomer = new Customer("Customer-1", "VIP");

        System.out.println("VIP Customer Price: " +
vipCustomer.getTicketPrice(ticket));  // Should apply 20% discount

        Customer regularCustomer = new Customer("Customer-2",
"Regular");
        System.out.println("Regular Customer Price: " +
regularCustomer.getTicketPrice(ticket));  // Full price
    }
}
```

---

33. Add Ticket Transfer Fee

Question:
Implement a transfer fee when tickets are transferred between customers. The fee should be 10% of the ticket price.

Solution:

- Calculate the transfer fee and deduct it from the final ticket value.

Code Implementation:

java
Copy code

```
package core;

public class TicketPool {

    public synchronized boolean transferTicket(Customer from, Customer
to, Ticket ticket) {
        double transferFee = ticket.getPrice() * 0.1; // 10% fee
        double finalPrice = ticket.getPrice() - transferFee;
```

```
        System.out.println("Transfer fee: " + transferFee + ", Final
Price: " + finalPrice);

        from.removeTicket(ticket);
        to.addTicket(ticket);
        return true;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import core.Ticket;
import core.Customer;

public class TicketTransferFeeTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        Ticket ticket = new Ticket("Ticket-1", "VIP", 100.0);
        Customer customerA = new Customer("CustomerA", "VIP");
        Customer customerB = new Customer("CustomerB", "Regular");

        pool.transferTicket(customerA, customerB, ticket);
    }
}
```

---

34. Add Multi-Tier Ticket System

Question:
Allow tickets to belong to different tiers (e.g., Bronze, Silver, Gold) with different prices.

Solution:

- Add a `tier` attribute to the `Ticket` class and adjust the price accordingly.

Code Implementation:

java
Copy code

```
package core;

public class Ticket {
    private final String id;
    private final String tier; // Bronze, Silver, Gold
    private double price;

    public Ticket(String id, String tier, double price) {
        this.id = id;
        this.tier = tier;
        this.price = price;

        switch (tier) {
            case "Gold":
                this.price *= 1.5;
                break;
            case "Silver":
                this.price *= 1.2;
                break;
            case "Bronze":
                this.price *= 1.0;
                break;
        }
    }

    public String getId() {
        return id;
    }

    public double getPrice() {
        return price;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;

public class MultiTierTicketTest {
    public static void main(String[] args) {
        Ticket goldTicket = new Ticket("Ticket-1", "Gold", 100.0);
        Ticket silverTicket = new Ticket("Ticket-2", "Silver", 100.0);
        Ticket bronzeTicket = new Ticket("Ticket-3", "Bronze", 100.0);

        System.out.println("Gold Ticket Price: " +
goldTicket.getPrice()); // 150.0
        System.out.println("Silver Ticket Price: " +
silverTicket.getPrice()); // 120.0
        System.out.println("Bronze Ticket Price: " +
bronzeTicket.getPrice()); // 100.0
    }
}
```

---

35. Implement Ticket Refund System

Question:
Allow customers to request a refund for tickets within a certain time period (e.g., 1 hour).

Solution:

- Track the ticket purchase time and implement a refund check.

Code Implementation:

java
Copy code

```
package core;
```

```
import java.time.Instant;

public class Ticket {
    private final String id;
    private final Instant purchaseTime;

    public Ticket(String id) {
        this.id = id;
        this.purchaseTime = Instant.now();
    }

    public boolean canRefund() {
        return
Instant.now().minusSeconds(3600).isBefore(purchaseTime); // 1 hour
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;

public class TicketRefundTest {
    public static void main(String[] args) throws InterruptedException
{
        Ticket ticket = new Ticket("Ticket-1");

        System.out.println("Can refund within 1 hour? " +
ticket.canRefund());

        // Simulate waiting for 2 hours
        Thread.sleep(7200000);  // 2 hours in milliseconds

        System.out.println("Can refund after 2 hours? " +
ticket.canRefund());
    }
```

```
}
```

---

36. Implement Ticket Lottery System

Question:
Create a lottery system where customers can randomly win tickets from a pool.

Solution:

- Use `Random` to pick a winner from a pool of customers.

Code Implementation:

java
Copy code

```
package core;

import java.util.List;
import java.util.Random;

public class TicketLottery {

    public Ticket drawWinner(List<Customer> customers) {
        Random random = new Random();
        Customer winner =
customers.get(random.nextInt(customers.size()));
        System.out.println("Winner: " + winner.getCustomerId());
        return new Ticket("Ticket-Winner");
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketLottery;
```

```java
import core.Customer;

import java.util.List;

public class TicketLotteryTest {
    public static void main(String[] args) {
        Customer customer1 = new Customer("Customer1");
        Customer customer2 = new Customer("Customer2");
        List<Customer> customers = List.of(customer1, customer2);

        TicketLottery lottery = new TicketLottery();
        lottery.drawWinner(customers);
    }
}
```

---

37. Add Customizable Ticket Prices

Question:
Allow administrators to set a custom price for tickets in the pool.

Solution:

- Add a `setPrice` method to the `Ticket` class that allows updating ticket prices.

Code Implementation:

java
Copy code

```java
package core;

public class Ticket {
    private final String id;
    private double price;

    public Ticket(String id, double price) {
        this.id = id;
        this.price = price;
```

```
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public double getPrice() {
        return price;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;

public class CustomTicketPriceTest {
    public static void main(String[] args) {
        Ticket ticket = new Ticket("Ticket-1", 100.0);
        System.out.println("Original Price: " + ticket.getPrice());

        ticket.setPrice(120.0);
        System.out.println("Updated Price: " + ticket.getPrice());
    }
}
```

---

38. Implement Ticket Validation

Question:
Validate that tickets have not been tampered with before they can be used.

Solution:

- Implement a simple validation using an ID hash to ensure the ticket hasn't been modified.

Code Implementation:

java
Copy code

```
package core;

import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;

public class Ticket {

    private final String id;

    public Ticket(String id) {
        this.id = id;
    }

    public String getId() {
        return id;
    }

    public boolean validateTicket(String ticketId) throws
NoSuchAlgorithmException {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(ticketId.getBytes());
        return ticketId.equals(new String(hash));
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;
```

```
public class TicketValidationTest {
    public static void main(String[] args) throws
NoSuchAlgorithmException {
        Ticket ticket = new Ticket("Ticket-1");
        System.out.println("Ticket Validation: " +
ticket.validateTicket("Ticket-1"));
    }
}
```

---

39. Add Dynamic Pricing Based on Time

Question:
Adjust ticket prices dynamically based on the time of day.

Solution:

- Implement time-based pricing by checking the system time.

Code Implementation:

java
Copy code

```
package core;

import java.time.LocalTime;

public class Ticket {
    private final String id;
    private double price;

    public Ticket(String id, double price) {
        this.id = id;
        this.price = price;
    }

    public double getPrice() {
        LocalTime now = LocalTime.now();
        if (now.isAfter(LocalTime.of(18, 0))) {  // After 6 PM
```

```
            return price * 1.2; // 20% increase in price
        }
        return price;
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.Ticket;

public class DynamicPricingTest {
    public static void main(String[] args) {
        Ticket ticket = new Ticket("Ticket-1", 100.0);
        System.out.println("Price: " + ticket.getPrice());
    }
}
```

---

40. Implement Ticket Pool Statistics Tracking

Question:
Track statistics such as total tickets added, removed, and currently in the pool.

Solution:

- Maintain counters for tickets added, removed, and remaining.

Code Implementation:

java
Copy code

```
package core;

public class TicketPool {
    private int totalAdded = 0;
```

```
    private int totalRemoved = 0;
    private int currentTickets = 0;

    public void addTicket(Ticket ticket) {
        totalAdded++;
        currentTickets++;
    }

    public void removeTicket() {
        totalRemoved++;
        currentTickets--;
    }

    public void printStats() {
        System.out.println("Total Tickets Added: " + totalAdded);
        System.out.println("Total Tickets Removed: " + totalRemoved);
        System.out.println("Current Tickets in Pool: " +
currentTickets);
    }
}
```

Test Case:

java
Copy code

```
package tests;

import core.TicketPool;
import core.Ticket;

public class TicketPoolStatsTest {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();
        pool.addTicket(new Ticket("Ticket-1", "VIP"));
        pool.addTicket(new Ticket("Ticket-2", "Regular"));
        pool.removeTicket();

        pool.printStats(); // Print statistics
    }}
```

Q&A with explanation

1. Implementing Ticket Pool Serialization

Question:
Modify the `TicketPool` class to allow saving and loading of tickets from a file using serialization.

Code Implementation:

java
Copy code

```
package core;

import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class TicketPool {

    private List<Ticket> tickets = new
ArrayList<>();

    public void addTicket(Ticket ticket) {
        tickets.add(ticket);
    }

    public void saveToFile(String filename) throws
IOException {
```

```
        try (ObjectOutputStream oos = new
ObjectOutputStream(new FileOutputStream(filename)))
{
            oos.writeObject(tickets);
        }
    }

    public void restoreFromFile(String filename)
throws IOException, ClassNotFoundException {
        try (ObjectInputStream ois = new
ObjectInputStream(new FileInputStream(filename))) {
            tickets = (List<Ticket>)
ois.readObject();
        }
    }

    public List<Ticket> getTickets() {
        return tickets;
    }
}
```

Explanation:

- **The `TicketPool` class has been modified to support serialization and deserialization using `ObjectOutputStream` and `ObjectInputStream`.**
- **The `saveToFile` method serializes the tickets into a file, and `restoreFromFile` allows you to load tickets back into the pool.**

- **This allows ticket pool state to persist across application restarts.**

---

2. Adding Customer Reservation with Expiry

Question:
Refactor the `Customer` class to allow reserving tickets for a period (e.g., 10 minutes).

Code Implementation:

java
Copy code

```java
package core;

import java.time.Instant;

public class Ticket {
    private final String id;
    private final Instant purchaseTime;

    public Ticket(String id) {
        this.id = id;
        this.purchaseTime = Instant.now();
    }

    public String getId() {
        return id;
```

```
    }

    public Instant getPurchaseTime() {
        return purchaseTime;
    }

    public boolean canReserve() {
        return
Instant.now().minusSeconds(600).isBefore(purchaseTi
me); // Can reserve within 10 minutes
    }
}

public class Customer {
    private final String customerId;

    public Customer(String customerId) {
        this.customerId = customerId;
    }

    public boolean reserveTicket(Ticket ticket) {
        if (ticket.canReserve()) {
            System.out.println(customerId + "
reserved " + ticket.getId());
            return true;
        } else {
```

```
            System.out.println(customerId + "
failed to reserve " + ticket.getId() + "
(expired)");
            return false;
        }
    }
}
```

Explanation:

- **The `Ticket` class includes a `purchaseTime` field and the `canReserve` method to check if the ticket can still be reserved (within 10 minutes).**
- **The `Customer` class now has a `reserveTicket` method that checks if a ticket can be reserved based on its expiry time.**

---

3. Customizing Ticket Pricing with Discounts

Question:
Enhance the `Ticket` class to support dynamic pricing based on the number of tickets in the pool.

Code Implementation:

java
Copy code

```
package core;
```

```
import java.util.List;

public class TicketPool {

    private List<Ticket> tickets;

    public TicketPool(List<Ticket> tickets) {
        this.tickets = tickets;
    }

    public synchronized double
getTicketPrice(Ticket ticket) {
        if (tickets.size() < 5) {
            return ticket.getPrice() * 0.9; // 10%
discount
        }
        if (tickets.size() > 50) {
            return ticket.getPrice() * 1.1; // 10%
price increase
        }
        return ticket.getPrice(); // Regular price
    }
}

public class Ticket {
    private final String id;
    private double price;
```

```
    public Ticket(String id, double price) {
        this.id = id;
        this.price = price;
    }

    public String getId() {
        return id;
    }

    public double getPrice() {
        return price;
    }
}
```

Explanation:

- **The `TicketPool` class checks the number of tickets in the pool and adjusts the price dynamically: tickets are 10% cheaper if there are fewer than 5 and 10% more expensive if there are more than 50 tickets.**
- **The `getPrice` method applies the appropriate pricing strategy.**

---

4. Implementing a Multi-Level Logging System

Question:
Modify the `Logger` class to support multiple log levels (INFO, ERROR, DEBUG, and WARN).

Code Implementation:

java
Copy code

```
package logging;

public enum LogLevel {
    INFO, ERROR, DEBUG, WARN
}

public class Logger {

    private LogLevel currentLevel = LogLevel.INFO;

    public void setLogLevel(LogLevel level) {
        this.currentLevel = level;
    }

    public void log(String message, LogLevel level)
{
        if (level.ordinal() >=
currentLevel.ordinal()) {
            System.out.println(level + ": " +
message);
        }
    }
}
```

Explanation:

- The `LogLevel` enum defines four log levels: INFO, ERROR, DEBUG, and WARN.
- The `Logger` class uses `setLogLevel` to change the log level, and the `log` method only logs messages if the message's log level is equal to or higher than the current level.

---

5. Handling Ticket Sales with Concurrency Control

Question:
Implement a thread-safe mechanism to ensure that customers cannot buy the same ticket at the same time (race condition).

Code Implementation:

java
Copy code

```
package core;

import java.util.List;

public class TicketPool {

    private final List<Ticket> tickets;

    public TicketPool(List<Ticket> tickets) {
        this.tickets = tickets;
```

```
    }

    public synchronized boolean buyTicket(Customer
customer, Ticket ticket) {
        if (tickets.contains(ticket)) {
            tickets.remove(ticket);

System.out.println(customer.getCustomerId() + "
successfully bought " + ticket.getId());
            return true;
        } else {

System.out.println(customer.getCustomerId() + "
failed to buy " + ticket.getId() + " (sold out)");
            return false;
        }
    }
}

public class Customer {
    private final String customerId;

    public Customer(String customerId) {
        this.customerId = customerId;
    }

    public String getCustomerId() {
        return customerId;
```

```
    }
}
```

Explanation:

- **The `buyTicket` method is synchronized to ensure that only one customer can purchase a ticket at any given time, preventing race conditions.**

---

6. Adding a VIP Customer Class with Special Privileges

Question:
Create a `VIPCustomer` subclass of the `Customer` class that gets priority for ticket purchases.

Code Implementation:

java
Copy code

```
package threads;

import core.Ticket;
import core.TicketPool;

public class Customer {
    private final String customerId;

    public Customer(String customerId) {
```

```
        this.customerId = customerId;
    }

    public String getCustomerId() {
        return customerId;
    }

    public boolean buyTicket(TicketPool pool,
Ticket ticket) {
        return pool.buyTicket(this, ticket);
    }
}

public class VIPCustomer extends Customer {

    public VIPCustomer(String customerId) {
        super(customerId);
    }

    @Override
    public boolean buyTicket(TicketPool pool,
Ticket ticket) {
        // VIP customer gets priority
        return pool.buyTicket(this, ticket);
    }
}
```

Explanation:

- **`VIPCustomer` extends `Customer`, and uses polymorphism to ensure VIP customers are processed first. In this case, both `Customer` and `VIPCustomer` call `buyTicket` but VIP customers might have a higher priority in future implementations.**

---

7. Implementing a Strategy for Ticket Filtering

Question:
Create an interface `TicketFilteringStrategy` with methods for filtering tickets by different criteria (e.g., type, price).

Code Implementation:

java
Copy code

```
package strategy;

import core.Ticket;

public interface TicketFilteringStrategy {
    boolean filter(Ticket ticket);
}

public class PriceFilteringStrategy implements
TicketFilteringStrategy {

    private double minPrice;
```

```
    private double maxPrice;

    public PriceFilteringStrategy(double minPrice,
double maxPrice) {
        this.minPrice = minPrice;
        this.maxPrice = maxPrice;
    }

    @Override
    public boolean filter(Ticket ticket) {
        return ticket.getPrice() >= minPrice &&
ticket.getPrice() <= maxPrice;
    }
}
```

Explanation:

- **The `TicketFilteringStrategy` interface allows for different filtering strategies, such as price filtering.**
- **Concrete implementations of the interface allow for filtering by price, type, etc.**

---

8. Implementing Ticket Expiry Notifications

Question:
Modify the `Ticket` class to support notifications when tickets are nearing their expiry date.

Code Implementation:

java
Copy code

```
package core;

import java.time.Instant;

public class Ticket {
    private final String id;
    private final Instant expirationTime;

    public Ticket(String id, Instant
expirationTime) {
        this.id = id;
        this.expirationTime = expirationTime;
    }

    public boolean isExpiringSoon() {
        return
Instant.now().plusSeconds(60).isAfter(expirationTim
e); // Within 1 minute of expiry
    }
}
```

Explanation:

- **`Ticket` class now includes an `expirationTime` and a method `isExpiringSoon` that checks if the ticket is close to expiration.**

---

9. GUI for Real-Time Ticket Management

Question:
Enhance the JavaFX GUI to allow customers to purchase and refund tickets in real-time.

Code Implementation:

java
Copy code

```
package ui;

import javafx.application.Application;
import javafx.application.Platform;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class TicketPurchaseGUI extends Application
{

    @Override
    public void start(Stage primaryStage) {
```

```
        Button purchaseButton = new
Button("Purchase Ticket");
        Button refundButton = new Button("Refund
Ticket");

        purchaseButton.setOnAction(event -> {
            // Purchase ticket logic
            System.out.println("Ticket
Purchased!");
        });

        refundButton.setOnAction(event -> {
            // Refund ticket logic
            System.out.println("Ticket Refunded!");
        });

        VBox vbox = new VBox(purchaseButton,
refundButton);
        Scene scene = new Scene(vbox, 300, 200);
        primaryStage.setScene(scene);
        primaryStage.setTitle("Ticket Management");
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Explanation:

- **This JavaFX GUI allows customers to purchase and refund tickets in real-time using buttons.**
- **Button actions print messages to the console for the time being, but could be expanded with logic to interact with the `TicketPool` class.**

---

10. Adding Ticket Queue for Waiting Customers

Question:
Implement a queue system for customers who are waiting for tickets.

Code Implementation:

java
Copy code

```
package core;

import java.util.concurrent.LinkedBlockingQueue;

public class TicketPool {
    private final LinkedBlockingQueue<Ticket>
ticketQueue = new LinkedBlockingQueue<>();

    public void addTicket(Ticket ticket) {
        ticketQueue.add(ticket);
```

```
    }

    public Ticket getNextTicket() {
        return ticketQueue.poll();
    }

    public void addWaitingCustomer(Customer
customer) {
        System.out.println(customer.getCustomerId()
+ " is waiting for a ticket.");
    }
}
```

Explanation:

- **The `TicketPool` class now uses a `LinkedBlockingQueue` for managing the tickets and a method for adding waiting customers if no tickets are available.**

11.Here's an implementation of enhancing customer behavior using polymorphism, where we introduce different customer types: `FastCustomer`, `SlowCustomer`, and `VIPCustomer`. Each type of customer interacts with the `TicketPool` in a unique way.

Code Implementation

Base Class (Customer.java)

java

Copy code

```
package core;


public abstract class Customer {

    protected String name;


    public Customer(String name) {

        this.name = name;

    }


    public abstract void retrieveTickets(TicketPool
pool, int count);


    public String getName() {

        return name;

    }

}
```

FastCustomer.java

java

Copy code

```
package core;


public class FastCustomer extends Customer {


    public FastCustomer(String name) {

        super(name);

    }


    @Override

    public void retrieveTickets(TicketPool pool,
int count) {

        for (int i = 0; i < count; i++) {

            Ticket ticket = pool.removeTicket();

            if (ticket != null) {

                System.out.println(name + " quickly
retrieved ticket: " + ticket.getId());

            } else {
```

```
                System.out.println(name + " found
no tickets available.");
                break;
            }
        }
    }
}
```

SlowCustomer.java

java

Copy code

```
package core;

public class SlowCustomer extends Customer {

    public SlowCustomer(String name) {
        super(name);
    }
```

```
    @Override
    public void retrieveTickets(TicketPool pool,
int count) {
        for (int i = 0; i < count; i++) {
            try {
                Thread.sleep(1000); // Simulate
slower retrieval
            } catch (InterruptedException e) {
                System.out.println(name + " was
interrupted while retrieving tickets.");
                return;
            }
            Ticket ticket = pool.removeTicket();
            if (ticket != null) {
                System.out.println(name + " slowly
retrieved ticket: " + ticket.getId());
            } else {
                System.out.println(name + " found
no tickets available.");
```

```
                    break;
                }
            }
        }
}
```

VIPCustomer.java

java

Copy code

```
package core;

public class VIPCustomer extends Customer {

    public VIPCustomer(String name) {
        super(name);
    }

    @Override
```

```
    public void retrieveTickets(TicketPool pool,
int count) {
        for (int i = 0; i < count; i++) {
            Ticket ticket = pool.removeTicket();
            if (ticket != null) {
                System.out.println(name + " (VIP)
retrieved ticket: " + ticket.getId());
            } else {
                System.out.println(name + " (VIP)
found no tickets available. VIP customers get
priority next time.");
                break;
            }
        }
    }
}
```

Ticket.java

java

Copy code

```java
package core;


public class Ticket {
    private final String id;

    public Ticket(String id) {
        this.id = id;
    }

    public String getId() {
        return id;
    }
}
```

TicketPool.java

java

Copy code

```java
package core;
```

```
import java.util.ArrayList;

import java.util.List;


public class TicketPool {

    private final List<Ticket> tickets = new
ArrayList<>();


    public synchronized void addTicket(Ticket
ticket) {

        tickets.add(ticket);

        System.out.println("Added ticket: " +
ticket.getId());

    }


    public synchronized Ticket removeTicket() {

        if (!tickets.isEmpty()) {

            return tickets.remove(0);

        }

        return null;
```

```
    }

    public synchronized int getTicketCount() {
        return tickets.size();
    }

    public synchronized void printTickets() {
        System.out.println("Tickets in the pool:");
        for (Ticket ticket : tickets) {
            System.out.println(ticket.getId());
        }
    }
}
```

Main.java

java

Copy code

```
package core;
```

```
public class Main {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();

        // Add tickets to the pool
        for (int i = 0; i < 10; i++) {
            pool.addTicket(new Ticket("Ticket-" +
i));
        }

        // Create customers
        Customer fastCustomer = new
FastCustomer("FastCustomer1");
        Customer slowCustomer = new
SlowCustomer("SlowCustomer1");
        Customer vipCustomer = new
VIPCustomer("VIPCustomer1");

        // Customers retrieve tickets
```

```
        fastCustomer.retrieveTickets(pool, 3); //
FastCustomer retrieves 3 tickets

        slowCustomer.retrieveTickets(pool, 4); //
SlowCustomer retrieves 4 tickets

        vipCustomer.retrieveTickets(pool, 5);  //
VIPCustomer retrieves 5 tickets


        // Print remaining tickets in the pool

        pool.printTickets();

        System.out.println("Total tickets remaining
in the pool: " + pool.getTicketCount());

    }

}
```

12. Implement Vendor as a Producer and Customer as a Consumer

Question:
Implement the producer-consumer pattern where vendors act as producers, adding tickets to the `TicketPool`, and customers act as consumers, retrieving tickets. Use `BlockingQueue` for thread-safe operations.

Code Implementation:

java

Copy code

```
package core;


import java.util.concurrent.BlockingQueue;

import java.util.concurrent.LinkedBlockingQueue;


public class TicketPool {

    private final BlockingQueue<Ticket> ticketQueue
= new LinkedBlockingQueue<>(50); // Max capacity of
50 tickets


    public void addTicket(Ticket ticket) throws
InterruptedException {

        ticketQueue.put(ticket); // Thread-safe
addition

        System.out.println("Produced: " +
ticket.getId());

    }
```

```
    public Ticket getTicket() throws
InterruptedException {
        Ticket ticket = ticketQueue.take(); //
Thread-safe retrieval
        System.out.println("Consumed: " +
ticket.getId());
        return ticket;
    }
}
```

Vendor (Producer):

java

Copy code

```
package core;


public class Vendor implements Runnable {
    private final TicketPool ticketPool;
    private final int ticketCount;
```

```
    public Vendor(TicketPool ticketPool, int
ticketCount) {
        this.ticketPool = ticketPool;
        this.ticketCount = ticketCount;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < ticketCount; i++) {
                Ticket ticket = new
Ticket("Ticket-" + i);
                ticketPool.addTicket(ticket);
                Thread.sleep(500); // Simulate
ticket production time
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Vendor
interrupted.");
```

```
        }
    }
}
```

Customer (Consumer):

java

Copy code

```
package core;


public class Customer implements Runnable {
    private final TicketPool ticketPool;
    private final int ticketCount;


    public Customer(TicketPool ticketPool, int
ticketCount) {
        this.ticketPool = ticketPool;
        this.ticketCount = ticketCount;
    }
```

```
    @Override
    public void run() {
        try {
            for (int i = 0; i < ticketCount; i++) {
                ticketPool.getTicket();
                Thread.sleep(1000); // Simulate
ticket consumption time
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Customer
interrupted.");
        }
    }
}
```

Main Class:

java

Copy code

```
package core;
```

```
public class Main {

    public static void main(String[] args) {

        TicketPool pool = new TicketPool();


        Thread vendorThread = new Thread(new
Vendor(pool, 10)); // Produces 10 tickets

        Thread customerThread = new Thread(new
Customer(pool, 10)); // Consumes 10 tickets


        vendorThread.start();

        customerThread.start();

    }

}
```

13.Handle Multiple Vendors and Customers

Question:
Extend the system to handle multiple vendors and customers running concurrently. Ensure tickets are shared properly among all customers.

Code Implementation: Update the Main class to handle multiple vendors and customers:

java

Copy code

```
package core;


public class Main {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();


        Thread vendor1 = new Thread(new
Vendor(pool, 10));
        Thread vendor2 = new Thread(new
Vendor(pool, 15));
        Thread customer1 = new Thread(new
Customer(pool, 12));
        Thread customer2 = new Thread(new
Customer(pool, 8));
```

```
        vendor1.start();
        vendor2.start();
        customer1.start();
        customer2.start();
    }
}
```

Explanation:

- Multiple producers (vendor1 and vendor2) add tickets to the pool.
- Multiple consumers (customer1 and customer2) retrieve tickets from the pool.
- The BlockingQueue ensures thread-safe access and proper coordination.

13. Handle Multiple Vendors and Customers

Question:
Extend the system to handle multiple vendors and customers running concurrently. Ensure tickets are shared properly among all customers.

Code Implementation: Update the Main class to handle multiple vendors and customers:

java

Copy code

```
package core;


public class Main {
    public static void main(String[] args) {
        TicketPool pool = new TicketPool();


        Thread vendor1 = new Thread(new
Vendor(pool, 10));
        Thread vendor2 = new Thread(new
Vendor(pool, 15));
        Thread customer1 = new Thread(new
Customer(pool, 12));
        Thread customer2 = new Thread(new
Customer(pool, 8));


        vendor1.start();
```

```
        vendor2.start();

        customer1.start();

        customer2.start();

    }

}
```

Explanation:

- **Multiple producers (vendor1 and vendor2) add tickets to the pool.**
- **Multiple consumers (customer1 and customer2) retrieve tickets from the pool.**
- **The BlockingQueue ensures thread-safe access and proper coordination.**

14. Add a Waiting Queue for Customers

Question:
If a customer tries to retrieve a ticket and none are available, they should wait until a ticket is produced.

Code Implementation:
This behavior is already managed by the take method

of BlockingQueue. The consumer thread (Customer) will block until a ticket becomes available.

Updated Customer (Consumer):

java

Copy code

```
package core;


public class Customer implements Runnable {
    private final TicketPool ticketPool;
    private final int ticketCount;

    public Customer(TicketPool ticketPool, int ticketCount) {
        this.ticketPool = ticketPool;
        this.ticketCount = ticketCount;
    }

    @Override
    public void run() {
```

```
        try {
            for (int i = 0; i < ticketCount; i++) {
                System.out.println("Attempting to
consume ticket...");
                ticketPool.getTicket();
                Thread.sleep(1000); // Simulate
ticket consumption time
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            System.out.println("Customer
interrupted.");
        }
    }
}
```

Explanation:

- When no tickets are available, the consumer thread (Customer) automatically waits until the producer adds tickets to the pool.

16. Add Statistics for Ticket Operations

Question:
Track and log the total number of tickets added and consumed by vendors and customers.

Code Implementation:

Updated TicketPool:

java

Copy code

```
package core;


import java.util.concurrent.BlockingQueue;

import java.util.concurrent.LinkedBlockingQueue;


public class TicketPool {

    private final BlockingQueue<Ticket> ticketQueue
= new LinkedBlockingQueue<>(50);

    private int totalProduced = 0;

    private int totalConsumed = 0;
```

```
    public synchronized void addTicket(Ticket
ticket) throws InterruptedException {

        ticketQueue.put(ticket);

        totalProduced++;

        System.out.println("Produced: " +
ticket.getId() + " | Total Produced: " +
totalProduced);

    }


    public synchronized Ticket getTicket() throws
InterruptedException {

        Ticket ticket = ticketQueue.take();

        totalConsumed++;

        System.out.println("Consumed: " +
ticket.getId() + " | Total Consumed: " +
totalConsumed);

        return ticket;

    }

}
```

Explanation:

- **The totalProduced and totalConsumed counters are incremented and logged whenever a ticket is added or removed.**